

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

October 14, 2025

Lecture 14

Good Papers

- Within 3 years
- 4 types of papers
- pick a topic or pick an application.
- Conferences:
 - ML: ICML, ICLR, NeurIPS.
 - AI: AAAI, IJCAI
 - Statistics: AISTATS, UAI
 - CV: CVPR, ICCV, ECCV, T-PAMI
 - NLP: EMNLP, ACL, NAACL, TACL
 - DM: WWW, KDD
- only main conference (long, short); No workshop.
- Present only. Present in a way that people know your problem, research landscape, key idea, key method, main experimental results.

Full Fine-tuning

- Simply continue training the LM on the output
- **Issue:** depending on optimizer, optimization method, can take lots of memory!
- **Example:** Training 65B parameter model with 16-bit mixed precision (Rajbhandari et al. 2019)

Model	65B parameters * 2b = 130GB	
	65B gradients * 2b = 130GB	
Optim- izer	65B parameters * 4b = 260GB	
	65B 1st-order * 4b = 260GB	
	65B 2nd-order * 4b = 260GB	
Activ- ations	Forward pass = 10-200GB	
	Backward pass = 10-200GB	

1000-1400GB of GPU memory!

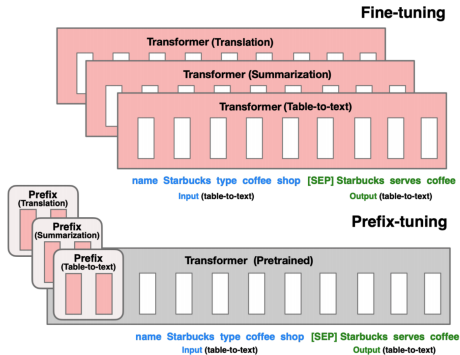
(can be reduced by using bfloat16, other optimizations)

Parameter-efficient Fine-tuning (PEFT)

- Don't tune all of the parameters, but just some!
 - Prompt/prefix tuning (last class)
 - Adapters
 - BitFit
 - LoRa

Prefix Tuning (Li and Liang 2021)

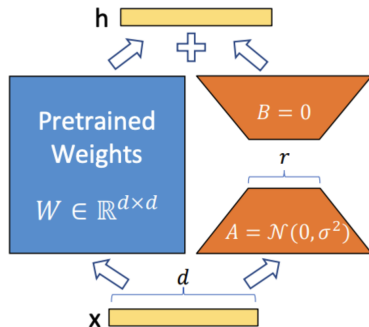
- "Prompt Tuning" optimizes only the embedding layer
- "Prefix Tuning" optimizes the prefix of all layers



LoRA

(Hu et al. 2021)

- Freeze pre-trained weights, train low-rank approximation of difference from pre-trained weights
- Advantage: after training, just add in to pre-trained weights — no new components!



RAG

Some Slides from Stanford CS25: V3 I Retrieval Augmented Language Models and

<https://www.phontron.com/class/anlp2024/assets/slides/anlp-10-rag.pdf>

Eliciting Outputs

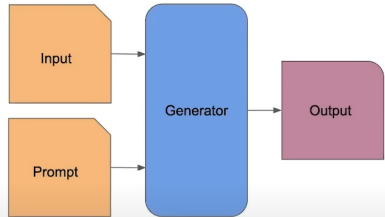
So we elicit specific outputs.

Problems:

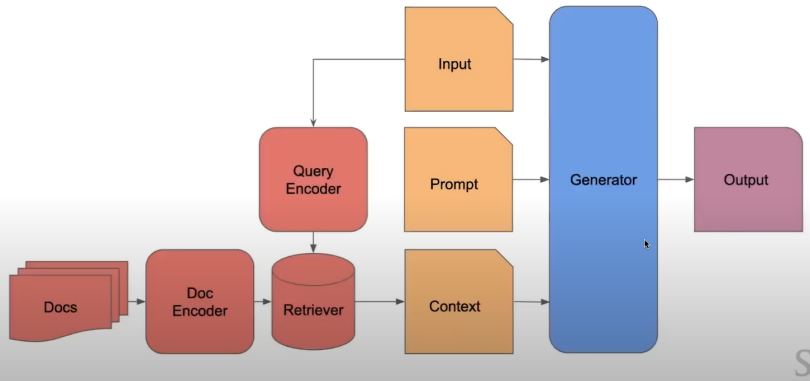
- Hallucination
- Attribution
- Staleness
- Revisions
- Customization

Solutions:

- Couple to external memory

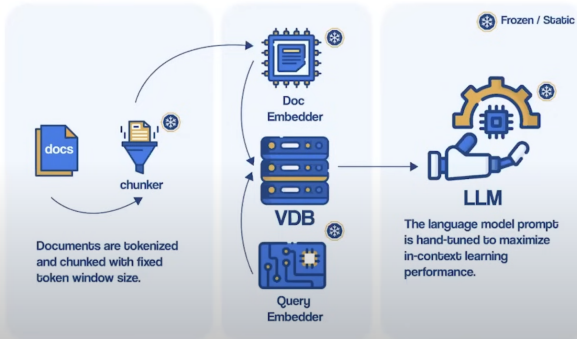


Contextualization



Frozen RAG

- No training: “In Context Learning” only

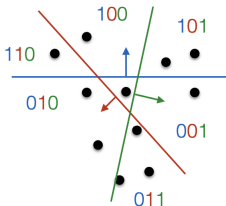


Approximate Nearest Neighbor Search

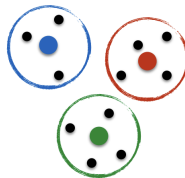
- Methods to retrieve embeddings in sub-linear time

Locality sensitive hashing:

make partitions in continuous space, use like inverted index

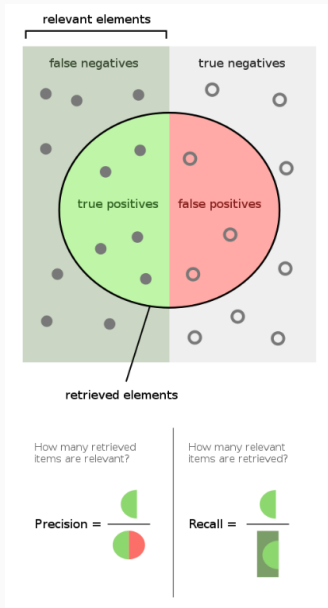


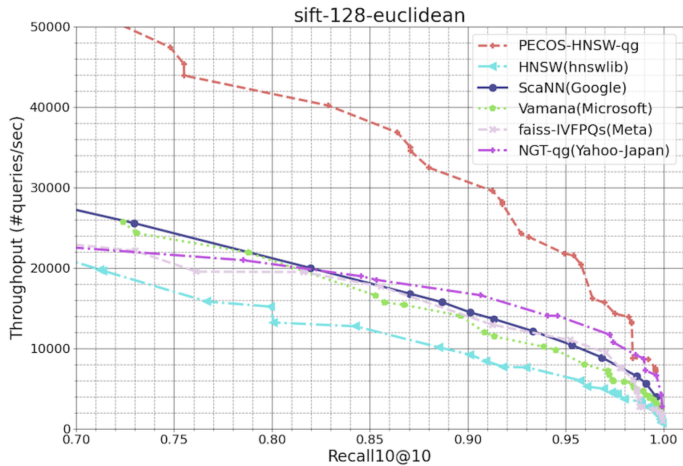
Graph-based search: create “hubs” and search from there



- Software: FAISS, ChromaDB

Precision, Recall, F1 = $\frac{2PR}{P+R}$





Sparse Retrieval

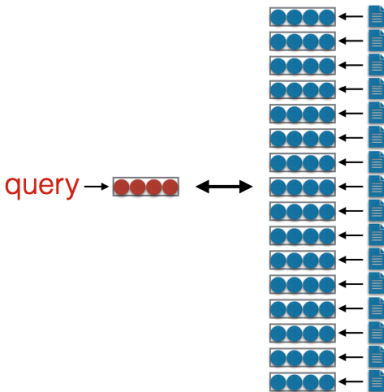
- Express the query and document as a sparse word frequency vector (usually normalized by length)

	$d_1 =$ what is life ? candy is life !	$d_2 =$ nlp is an acronym for natural language processing	$d_3 =$ I like to do good research on nlp
q=what is nlp			
what	$\begin{pmatrix} 0.33 \\ 0 \\ 0.33 \\ 0.33 \\ 0 \\ \dots \end{pmatrix}$	$\begin{pmatrix} 0.25 \\ 0.125 \\ 0 \\ 0.25 \\ 0 \\ \dots \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 0.125 \\ 0 \\ 0 \\ \dots \end{pmatrix}$
candy			
nlp			
is			
language			
...			
	$q \cdot d_1 = 0.165$	$q \cdot d_2 = 0.0825$	$q \cdot d_3 = 0.0413$

- Find the document with the highest inner-product or cosine similarity in the document collection

Dense Retrieval

- Encode document/query and find nearest neighbor
- Can use:
 - Out-of-the-box embeddings
 - Learned embeddings



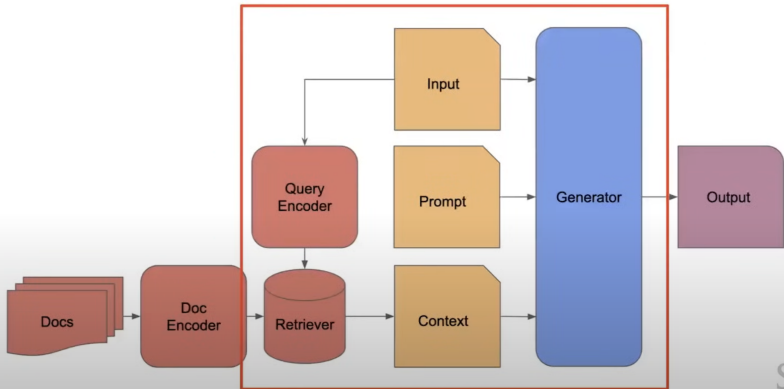
Learning Retrieval-oriented Embeddings

- Select positive and negative documents, train using a contrastive loss (e.g. hinge loss)

$$\mathcal{L}(\theta, q) = \sum_{d_{\text{pos}} \in D_{\text{pos}}} \sum_{d_{\text{neg}} \in D_{\text{neg}}} \max(0, s(q, d_{\text{neg}}; \theta) - s(q, d_{\text{pos}}; \theta))$$

- **DPR** (Karpukhin et al. 2020): learn encoders based on a BM25 hard negatives and in-batch negatives.
- **Contriever** (Izacard et al. 2022): contrastive learning using two random spans as positive pairs

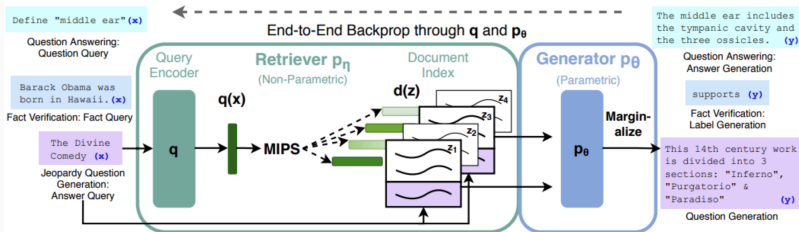
Contextualization of both



Retriever + Generator End-to-end Training (“RAG”)

(Lewis et al. 2020)

- Train the retriever and reader to improve accuracy
- **Reader:** Maximize generation likelihood given single retrieved document
- **Retriever:** Maximize overall likelihood by optimizing mixture weights over documents



End-to-end Training Equations

(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} \underbrace{p_{\eta}(z|x)}_{\text{Retriever}} \underbrace{p_{\theta}(y_i|x, z, y_{1:i-1})}_{\text{Generator}}$$

- Probability of the retriever is based on embeddings

$$p_{\eta}(z|x) \propto \exp(\mathbf{d}(z)^{\top} \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

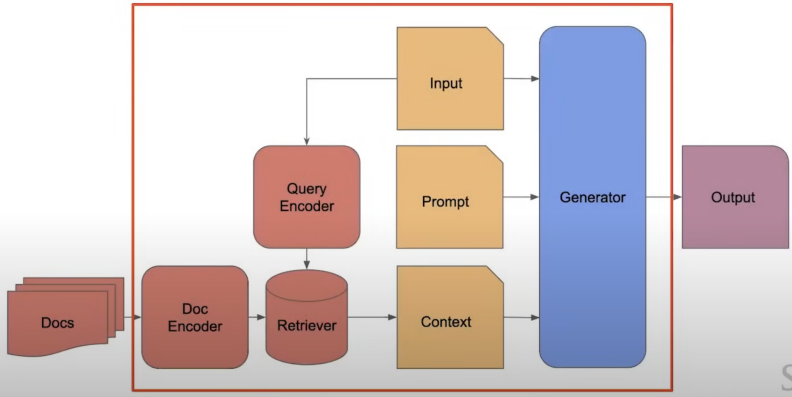
- Adjusts retriever to give higher similarities helpful docs
- Issue: search index becomes stale $\rightarrow$ can only train $q(x)$

Whole point of RAG: Freezing Suboptimal!

Table 6: Ablations on the dev set. As FEVER is a classification task, both RAG models are equivalent.

Model	NQ	TQA Exact Match	WQ	CT	Jeopardy-QGen B-1 QB-1		MSMarco R-L B-1		FVR-3 Label Accuracy	FVR-2
RAG-Token-BM25	29.7	41.5	32.1	33.1	17.5	22.3	55.5	48.4	75.1	91.6
RAG-Sequence-BM25	31.8	44.1	36.6	33.8	11.1	19.5	56.5	46.9		
RAG-Token-Frozen	37.8	50.1	37.1	51.1	16.7	21.7	55.9	49.4	72.9	89.4
RAG-Sequence-Frozen	41.2	52.1	41.8	52.6	11.8	19.6	56.7	47.3		
RAG-Token	43.5	54.8	46.5	51.9	17.9	22.6	56.2	49.4	74.5	90.6
RAG-Sequence	44.0	55.8	44.9	53.4	15.3	21.5	57.2	47.5		

Contextualization all the way



REALM (Guu et al 2020)

- The OG of non-frozen dense retrieval augmented LMs.
- Backprop all the way, async updates.
- Updating the document encoder is hard, can you see why?
- Really visionary work.
- Downside: not really generative, BERT all the way.

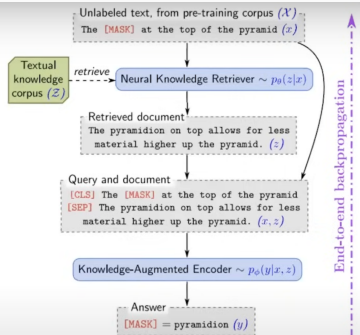


Figure 1. REALM augments language model pre-training with a **neural knowledge retriever** that retrieves knowledge from a **textual knowledge corpus**, Z (e.g., all of Wikipedia). Signal from the language modeling objective backpropagates all the way through the retriever, which must consider millions of documents in Z —a significant computational challenge that we address.

When do we Retrieve?

- **Once, at the beginning of generation**
 - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
 - Generate a search token (Schick et al. 2023)
 - Search when the model is uncertain (Jiang et al. 2023)
- **Every token**
 - Find similar final embeddings (Khandelwal et al. 2019)

Triggering Retrieval w/ Tokens

- Toolformer (Schick et al. 2023) generates tokens that trigger retrieval (or other tools)
- Training is done in an iterative manner - generate and identify successful retrievals

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

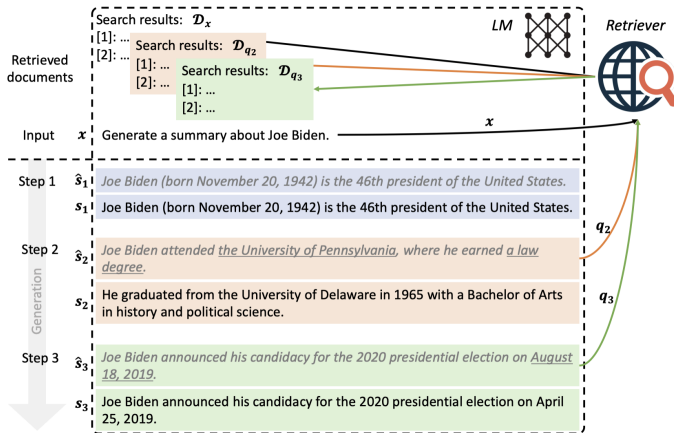
Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Triggering Retrieval w/ Uncertainty

- FLARE (Jiang et al. 2023) tries to generate content, then does retrieval if LM certainty is low



Token-level Softmax Modification

- kNN-LM (Khandelwal et al. 2019) retrieves similar examples, and uses the following token from them

